Unit 1: Introduction to Problem Solving

1. **Define the following terms: compiler, assembler, interpreter, debugger**
   - **Compiler**: A program that translates high-level code (like C, Java) into machine language or intermediate code. It translates the entire program at once.
   - **Assembler**: A program that converts assembly language into machine language (binary code).
   - **Interpreter**: A program that translates high-level code into machine language but does it line-by-line, rather than all at once.
   - **Debugger**: A tool used to test and debug (find errors in) programs by providing functionalities like breakpoints, step execution, and variable monitoring.

2. **Define algorithm.**
   - An **algorithm** is a step-by-step procedure or formula for solving a problem. It is a finite set of instructions that provides a solution to a specific task or problem.

3. **List the characteristics of an algorithm.**
   - **Finiteness**: The algorithm must have a finite number of steps.
   - **Definiteness**: Each step must be clear and unambiguous.
   - **Input**: It should take zero or more inputs.
   - **Output**: It should produce one or more outputs.
   - **Effectiveness**: Each step must be basic enough to be carried out practically within a finite amount of time.

4. **Write any two advantages of an algorithm.**
   - **Clarity**: Algorithms help break down problems into clear and manageable steps.
   - **Efficiency**: They allow for analyzing performance and optimizing the solution process.

5. **Write any two limitations of an algorithm.**
   - **Not always optimal**: Algorithms may not always provide the most efficient solution.
   - **Language-independent**: Although an advantage in some cases, this makes it harder to directly implement.

6. **Define flowchart.**
   - A **flowchart** is a diagrammatic representation of an algorithm. It uses symbols and arrows to show the flow of steps required to complete a task.

7. **What is assembly language?**
   - **Assembly language** is a low-level programming language that uses symbolic names (like `MOV`, `ADD`, `SUB`) instead of binary code. It is closer to machine language and is specific to hardware architecture.

8. **What is machine language?**
   - **Machine language** is the lowest-level programming language consisting of binary code (0s and 1s) that the computer's central processing unit (CPU) can directly execute.

9. **Write any two advantages of flowchart.**
   - **Visualization**: Flowcharts provide a visual representation of the logic of a system.
   - **Communication**: They are a great tool for communicating the flow of a program among team members.

10. **Write any two limitations of flowchart.**
    - **Complexity**: For large systems, flowcharts can become complex and difficult to understand.
    - **Time-consuming**: Drawing detailed flowcharts is a time-consuming task.

11. **What is low-level Language? Write any 2 advantages of it.**
    - **Low-level language** is a programming language that is close to machine code and provides little abstraction from the hardware.
    - **Advantages**:
      - **Efficiency**: Programs written in low-level languages run faster as they are closely related to machine code.
      - **Control**: It provides more control over hardware components.

12. **What is high-level Language? Write any 2 advantages of it.**
   - **High-level language** is a programming language that is user-friendly and abstracted from the hardware (e.g., Python, Java).
   - **Advantages**:
     - **Ease of use**: High-level languages are easier to learn, read, and write.
     - **Portability**: Programs written in high-level languages can often run on different types of machines with minimal changes.

13. **Write advantages and Limitations of low-level language.**
   - **Advantages**:
     - **Speed**: Programs run faster due to minimal abstraction from hardware.
     - **Memory efficiency**: Low-level languages allow direct memory manipulation, optimizing resource usage.
   - **Limitations**:
     - **Complexity**: Writing and debugging in low-level languages is difficult due to the lack of readability.
     - **Non-portability**: Programs are hardware-specific, requiring rewriting for different machines.

---Let's continue with **Unit 1: Introduction to Problem Solving** and then proceed to the other units.

Unit 1: Introduction to Problem Solving (Continued)

14. **What is problem solving? Write problem-solving steps.**
   - **Problem-solving** is the process of finding solutions to complex issues.
   - **Steps**:
     1. **Understanding the problem**: Analyze the problem and clarify the requirements.
     2. **Plan the solution**: Develop an algorithm or strategy to solve the problem.
     3. **Design the solution**: Translate the plan into a structure like flowcharts or pseudocode.
     4. **Implementation**: Code the solution.
     5. **Test the solution**: Verify that the solution works as expected.
     6. **Optimization**: Refine the solution for better performance.

15. **Define algorithm? Explain its characteristics with a proper example.**
   - An **algorithm** is a step-by-step method for solving a problem.
   - **Characteristics**:
     - **Finiteness**: It must terminate after a finite number of steps.
     - **Definiteness**: Each step should be precisely defined.
     - **Input**: Takes input(s) to process.
     - **Output**: Produces an output(s).
     - **Effectiveness**: Operations should be basic enough to be carried out by a machine.
   - **Example**: Algorithm to add two numbers:
     1. Start
     2. Input two numbers
     3. Add the two numbers
     4. Display the result
     5. End

16. **Define Flowchart? Describe notations to draw a flowchart with proper example.**
   - A **flowchart** is a graphical representation of an algorithm.
   - **Notations**:
     - **Oval**: Start/End
     - **Parallelogram**: Input/Output
     - **Rectangle**: Process (e.g., computations)
     - **Diamond**: Decision (e.g., if-else)
     - **Arrow**: Flow of control

- **Example**: Flowchart to add two numbers (uses input, process, and output notations).

17. **State advantages and limitations of algorithm.**
- **Advantages**:
  - **Clarity**: Helps break down a problem into structured steps.
  - **Debugging**: Easier to troubleshoot and modify.
- **Limitations**:
  - **Time-consuming**: Writing a detailed algorithm can take time.
  - **Language dependency**: Algorithms must eventually be converted into code.

18. **State advantages and limitations of flowchart.**
- **Advantages**:
  - **Visualization**: Makes complex processes easier to understand.
  - **Problem Analysis**: Provides a clear picture of problem structure.
- **Limitations**:
  - **Complexity**: Large processes result in complex flowcharts.
  - **Updates**: Modifying a flowchart can be cumbersome.

19. **Compare algorithm with flowchart.**
- **Algorithm**: Text-based, step-by-step procedure for solving a problem.
- **Flowchart**: Graphical representation using symbols to depict the flow of a process.
- **Comparison**:
  - **Ease of understanding**: Flowcharts are easier to interpret visually.
  - **Application**: Algorithms are more structured for textual or coding representation.

20. **What is Pseudo code? State advantages and limitations of it?**
- **Pseudo code**: A high-level description of an algorithm in plain language, resembling programming language but not following syntax rules strictly.
- **Advantages**:
  - Easy to write and understand.
  - Helps in planning the logic without worrying about specific syntax.
- **Limitations**:
  - Not executable on a computer.
  - Can sometimes be ambiguous.

21. **What is Programming Language? Explain its type?**
- **Programming language**: A formal language used to write instructions that a computer can execute.
- **Types**:
  - **Low-level language**: Close to machine code (e.g., Assembly).
  - **High-level language**: Abstracted from machine code (e.g., Python, Java).

22. **Practice on drawing algorithm and flowchart.**
- **Exercise**: Draw an algorithm and flowchart to solve simple problems like calculating the sum of two numbers or finding the largest of three numbers.

Unit 2: Introduction to Python

1. **What is indentation?**
- **Indentation** refers to spaces or tabs used at the beginning of lines in Python to define blocks of code, such as functions, loops, and conditional statements.

2. **What is a slice operator?**

- The **slice operator** (`:`) is used to extract parts of a sequence like strings, lists, or tuples by specifying a start and end index. Example: `s[1:4]` extracts a portion from index 1 to 3.

3. **What is comment?**
   - A **comment** is a line in code ignored by the interpreter and used to explain the code for better readability. In Python, comments start with `#`.

4. **Give the use of `index()` in a string.**
   - The `index()` function returns the position (index) of the first occurrence of a substring in a string. Example: `"hello".index('e')` returns `1`.

5. **What is the use of `if` statement?**
   - The `if` statement is used for decision-making by executing a block of code if a condition is `True`.

6. **What is the use of `break` statement?**
   - The `break` statement is used to exit from a loop prematurely, ignoring the rest of the loop's code even if the loop condition is still true.

7. **What is the use of `continue` statement?**
   - The `continue` statement skips the current iteration of a loop and moves to the next iteration.

8. **What is the use of `pass` statement?**
   - The `pass` statement is a null operation. It is used when a statement is syntactically required but no action is needed.

9. **What is mutable?**
   - **Mutable** objects are those that can be modified after their creation. Examples: lists, dictionaries, sets.

10. **What is immutable?**
    - **Immutable** objects are those that cannot be modified after their creation. Examples: strings, tuples.

11. **What is string?**
    - A **string** is a sequence of characters enclosed in single, double, or triple quotes in Python. Example: `"Hello"` or `'World'`.

12. **What is list?**
    - A **list** is a mutable, ordered collection of items, defined using square brackets. Example: `[1, 2, 3, 4]`.

13. **What is tuple?**
    - A **tuple** is an immutable, ordered collection of items, defined using parentheses. Example: `(1, 2, 3, 4)`.

14. **What is set?**
    - A **set** is an unordered collection of unique items, defined using curly brackets. Example: `{1, 2, 3, 4}`.

15. **What is dictionary?**
    - A **dictionary** is a mutable collection of key-value pairs, defined using curly brackets. Example: `{'key': 'value'}`.

Continuing from **Unit 2: Introduction to Python**, let's finish the remaining questions and proceed with the other units.

Unit 2: Introduction to Python (Continued)

16. **What is a function?**
    - A **function** is a block of reusable code that performs a specific task. It can accept inputs (arguments) and return outputs (results).

17. **Write the syntax to define a function.**
```python
def function_name(parameters):
    # function body
    return value
```

18. **What is an anonymous function?**
- An **anonymous function** in Python is a function defined without a name using the `lambda` keyword. Example:
```python
lambda x: x + 2
```

19. **List the features of Python.**
- **Interpreted**: Python code is executed line by line.
- **Dynamically typed**: Variables don't need to be declared with a type.
- **High-level**: Python is user-friendly and abstracted from hardware.
- **Large standard library**: Offers built-in modules and functions.
- **Object-oriented**: Supports concepts like classes and inheritance.

20. **How are comments written in Python?**
- Single-line comments are written with `#` and multi-line comments can be written using triple quotes (`"` or `"""`).

21. **Compare list and tuple (any two points).**
- **Mutability**: Lists are mutable, while tuples are immutable.
- **Syntax**: Lists use square brackets (`[]`), while tuples use parentheses (`()`).

22. **Write code to print the elements of the list `l1 = [10, 20, 30, 40, 50]`.**
```python
l1 = [10, 20, 30, 40, 50]
for i in l1:
    print(i)
```

23. **Explain the `range()` function and its parameters.**
- The `range()` function generates a sequence of numbers. Syntax: `range(start, stop, step)`.
  - **start**: Beginning of the sequence (default is 0).
  - **stop**: End of the sequence (not inclusive).
  - **step**: Difference between each number in the sequence (default is 1).

24. **What is the difference between `pop()` and `del` in a list?**
- **`pop()`**: Removes the last element (or element at a specified index) from a list and returns it.
- **`del`**: Deletes an element at a specified index without returning it.

25. **How to add multiple elements at the end of the list?**
- Use the `extend()` method to add multiple elements to a list.
```python
l1 = [1, 2, 3]
l1.extend([4, 5, 6])
```

26. **What is a function? Write the syntax to define a function.**
- Same as Question 16 and 17 above.

27. **Write a note on Python data types.**
   - Python has several data types:
     - **Numeric**: int, float, complex
     - **Sequence**: list, tuple, range
     - **Text**: str
     - **Set**: set, frozenset
     - **Mapping**: dict
     - **Boolean**: bool

28. **Explain slicing concept in Python.**
   - **Slicing** allows you to extract a subset of a sequence (string, list, tuple) by specifying start, stop, and step indices.
   Example: `s[1:4]` extracts elements from index 1 to 3.

29. **Explain any four string methods.**
   - **`upper()`**: Converts all characters in a string to uppercase.
   - **`lower()`**: Converts all characters to lowercase.
   - **`replace(old, new)`**: Replaces all occurrences of a substring with another.
   - **`split()`**: Splits the string into a list based on a separator.

30. **Explain `while` loops with an example.**
   - A **`while` loop** repeats a block of code as long as the condition is true.
   Example:
   ```python
   i = 1
   while i < 5:
       print(i)
       i += 1
   ```

31. **Explain `for` loops with an example.**
   - A **`for` loop** is used to iterate over a sequence.
   Example:
   ```python
   for i in [1, 2, 3]:
       print(i)
   ```

32. **Explain the following statements:**
   - **if**: Executes a block of code if a condition is true.
   - **if-else**: Executes one block of code if a condition is true, another block if it's false.
   - **while**: Repeats a block of code as long as a condition is true.
   - **for**: Iterates over a sequence.

33. **How to perform input-output operations? Explain with example.**
   - **Input**: Use `input()` to take input from the user.
   - **Output**: Use `print()` to display output.
   Example:
   ```python
   name = input("Enter your name: ")
   print("Hello", name)
   ```

34. **What are the usage of `tuple()`, `count()`, and `index()` functions of a tuple?**

- **`tuple()`**: Converts an iterable into a tuple.
- **`count()`**: Returns the number of occurrences of a value in a tuple.
- **`index()`**: Returns the index of the first occurrence of a value in a tuple.

35. **What are the usage of dictionary methods `copy()`, `get()`, `items()`, and `keys()`?**
- **`copy()`**: Returns a shallow copy of the dictionary.
- **`get(key)`**: Returns the value for a specified key.
- **`items()`**: Returns a view object of key-value pairs.
- **`keys()`**: Returns a view object of the dictionary's keys.

36. **Explain union, intersection, difference, and symmetric difference of sets with examples.**
- **Union**: Combines elements from two sets. `A | B`.
- **Intersection**: Returns common elements in both sets. `A & B`.
- **Difference**: Returns elements in one set but not in the other. `A - B`.
- **Symmetric Difference**: Returns elements in either set, but not both. `A ^ B`.

Example:
```python
A = {1, 2, 3}
B = {3, 4, 5}
print(A | B)  # Union: {1, 2, 3, 4, 5}
print(A & B)  # Intersection: {3}
print(A - B)  # Difference: {1, 2}
print(A ^ B)  # Symmetric difference: {1, 2, 4, 5}
```

37. **Explain the syntax to define and call a function with a proper example.**
```python
def greet(name):
    return "Hello " + name

# Call the function
print(greet("John"))
```

38. **Write a note on function arguments.**
- Functions can accept various types of arguments:
  - **Positional arguments**: Passed in the order defined.
  - **Keyword arguments**: Passed with a key-value pair.
  - **Default arguments**: Have a default value if none is provided.
  - **Variable-length arguments**: Accept an arbitrary number of arguments using `*args` or `**kwargs`.

39. **Explain global and local variables with examples.**
- **Local variables** are declared inside a function and are only accessible within that function.
- **Global variables** are declared outside any function and are accessible throughout the program.

Example:
```python
x = 10  # Global variable

def my_function():
    x = 5  # Local variable
    print(x)  # Outputs 5

my_function()
print(x)  # Outputs 10 (global variable)
```

```

40. **What is anonymous function or lambda function? Explain with example.**
   - An **anonymous function** is a function without a name, defined using `lambda` in Python.
   Example:
   ```python
   square = lambda x: x * 2
   print(square(5))  # Outputs 10
   ```

41. **Python programs using functions.**
   - This is an exercise-based question. Write simple Python programs to demonstrate the use of functions, such as a program to calculate the factorial of a number using a function.

Unit 3: Classes, Objects, and Inheritance

1. **What is class?**
   - A **class** is a blueprint for creating objects, providing initial values for state (member variables) and implementations of behavior (member functions or methods).

2. **What is object?**
   - An **object** is an instance of a class. It contains both data and methods that act on that data.

3. **What is inheritance?**
   - **Inheritance** is a feature of OOP that allows a new class (child) to inherit attributes and methods from

 an existing class (parent).

4. **What is hybrid inheritance?**
   - **Hybrid inheritance** is a combination of more than one type of inheritance, such as single, multiple, or hierarchical inheritance in a single program.

5. **What is hierarchical inheritance?**
   - **Hierarchical inheritance** occurs when multiple child classes inherit from a single parent class.

6. **What is class? Write syntax to create a class.**
   ```python
   class MyClass:
       def __init__(self, value):
           self.value = value
   ```

7. **What is object? Write syntax to create an object.**
   ```python
   obj = MyClass(10)
   ```

8. **What is inheritance? Write its advantages.**
   - **Inheritance** allows code reusability, easy maintenance, and a logical class hierarchy.

9. **Write syntax for single inheritance.**
   ```python

```
class Parent:
    pass

class Child(Parent):
    pass
```

10. **What are different kinds of relationships held by hierarchical inheritance?**
    - In **hierarchical inheritance**, there are parent-child relationships, where the child class can inherit properties and behaviors from the parent class.

11. **Demonstrate how to create a class with a suitable example.**
    ```python
    class Car:
        def __init__(self, model):
            self.model = model

        def display(self):
            print("Car model:", self.model)

    car = Car("Toyota")
    car.display()
    ```

12. **What is inheritance? Explain its types.**
    - **Single Inheritance**: One child class inherits from one parent class.
    - **Multiple Inheritance**: A child class inherits from more than one parent class.
    - **Multilevel Inheritance**: A child class inherits from a parent class, which itself inherits from another parent class.
    - **Hierarchical Inheritance**: Multiple child classes inherit from a single parent class.
    - **Hybrid Inheritance**: A combination of multiple types of inheritance.

13. **Explain single, multiple, multilevel, and hierarchical inheritance.**
    - **Single Inheritance**: One class inherits from one parent.
    - **Multiple Inheritance**: One class inherits from more than one parent class.
    - **Multilevel Inheritance**: A class inherits from a child class of another class.
    - **Hierarchical Inheritance**: Multiple classes inherit from a single parent class.

14. **Python programs using class.**
    - Example programs where you define a class and create objects to call methods and access properties.

15. **Python programs using inheritance.**
    - Example programs where you demonstrate different types of inheritance in Python.

Unit 4: Modules and Packages

1. **What is module?**
    - A **module** is a file containing Python definitions and statements. A module allows for code reuse by grouping related functions and variables.

2. **List any 4 built-in modules used in Python.**
    - **`math`**: Provides mathematical functions.
    - **`os`**: Provides functions to interact with the operating system.

- **`sys`**: Provides access to system-specific parameters and functions.
- **`random`**: Provides functions to generate random numbers.

3. **Explain any 4 methods of `math` module.**
   - **`sqrt()`**: Returns the square root of a number.
   - **`ceil()`**: Rounds a number up to the nearest integer.
   - **`floor()`**: Rounds a number down to the nearest integer.
   - **`pow()`**: Returns the result of a number raised to the power of another.

4. **What is package? List its types.**
   - A **package** is a collection of modules. Types:
     - **Built-in packages**: Provided by Python, like `os`, `sys`, etc.
     - **User-defined packages**: Created by the user to organize their code.

5. **What is module? Write syntax to create and import a module in a Python program.**
   - A module is a file containing Python code (functions, classes, etc.). To create and import a module:
   ```python
   # mymodule.py
   def greet():
       print("Hello!")

   # Importing the module
   import mymodule
   mymodule.greet()
   ```

6. **What is a user-defined package? Write syntax to create and import it in Python.**
   - A **user-defined package** is a directory containing multiple modules and an `__init__.py` file. Example:
   ```python
   # Create a directory structure
   # mypackage/
   #     __init__.py
   #     module1.py
   ```

7. **Write a program to demonstrate how to create and import a module.**
   - See answer to Question 5.

8. **Write a program to demonstrate how to create and import a user-defined package.**
   ```python
   # File: mypackage/module1.py
   def display():
       print("This is module1.")

   # File: main.py
   from mypackage import module1
   module1.display()
   ```